Linux Ext4 Internals: Architecture, Features, and Performance Tuning

Day 3: Data Allocation & I/O Path

Duration: 3.5 hours | **Level:** Expert

Instructor: Chandrashekar Babu
training@chandrashekar.info

Website:
https://www.chandrashekar.info/
https://www.slashprog.com/

Day 3 Agenda: What We'll Cover Today

Module 5: Extent-Based Allocation (1.5 hours)

- Extent tree operations (insert, delete, merge, split)
- Delayed allocation and multiblock allocator (mballoc)
- Fragmentation avoidance and defragmentation
- Low-memory and ENOSPC handling
- Orphan inode cleanup

Module 6: I/O Path & Memory Integration (1.5 hours)

- VFS integration (file_operations, address_space_ops)
- Page cache and writeback mechanisms
- Direct I/O vs Buffered I/O
- Read-ahead, AIO, and io_uring
- Folio interface and memory management
- Zoned device support
- 4KB vs 16KB page alignment

Hands-On Labs (30 minutes)

- I/O tracing with blktrace, perf, bpftrace
- Buffered vs Direct I/O experiments
- Page cache pressure analysis

Break: 10 minutes between modules

Module 5 Overview: Extent-Based Allocation Architecture

Evolution from Indirect Blocks

Indirect Blocks (Ext2/3)

```
┌──────────────────────────────────────────────────────────────┐
│ Inode                                                        │
│ i_block[0..11] → Direct blocks                               │
│ i_block[12]    → Single indirect                             │
│ i_block[13]    → Double indirect                             │
│ i_block[14]    → Triple indirect                             │
└──────────────────────────────────────────────────────────────┘
```

Limitations:

- Many seeks for large files
- Fragmentation
- 4-level indirection overhead

Extents (Ext4)

```
┌──────────────────────────────────────────────────────────────┐
│ Inode                                                        │
│ Extent Header                                                │
│  ├─── Extent: [0-127]   → blocks 1000-1127                   │
│  ├─── Extent: [128-255] → blocks 2000-2127                   │
│  └─── Extent: [256-383] → blocks 3000-3127                   │
└──────────────────────────────────────────────────────────────┘
```

Benefits:

- One extent = 128MB contiguous space (4KB blocks)
- O(log n) lookup
- Less metadata

Extent Tree Data Structures: Core Structures

```
// Extent header (at start of each extent block)
struct ext4_extent_header {
    __le16 eh_magic;       /* Magic number: 0xF30A */
    __le16 eh_entries;     /* Number of valid entries */
    __le16 eh_max;         /* Capacity of entries */
    __le16 eh_depth;       /* Tree depth (0 = leaf) */
    __le32 eh_generation;  /* Generation counter */
};

// Leaf extent (points to physical blocks)
struct ext4_extent {
    __le32 ee_block;       /* First logical block */
    __le16 ee_len;         /* Number of blocks (1-32768) */
    __le16 ee_start_hi;    /* High 16 bits of physical block */
    __le32 ee_start_lo;    /* Low 32 bits of physical block */
};

// Index extent (points to next level)
struct ext4_extent_idx {
    __le32 ei_block;       /* Logical block covered */
    __le32 ei_leaf_lo;     /* Pointer to leaf block (low) */
    __le16 ei_leaf_hi;     /* Pointer to leaf block (high) */
    __le16 ei_unused;
};
```

Max Extent Length

32768 blocks (128MB with 4KB blocks)

Max Tree Depth

5 levels (supports up to 16TB files)

Max Entries per Block

340 extents (with 4KB block size)

Extent Tree Operations: Lookup

Finding Blocks in Extent Tree

```
// Main lookup function
static int ext4_ext_find_extent(struct inode *inode, ext4_lblk_t block,
 struct ext4_ext_path *path) {
 struct ext4_extent_header *eh;
 struct ext4_extent_idx *idx;
 int depth;

 // Get root header
 eh = ext4_inode_ext_header(inode);
 depth = eh->eh_depth;
 path[0].p_hdr = eh;

 // Traverse down the tree
 for (i = 0; i <= depth; i++) {
 if (i == depth) {
 // Leaf level - get extent
 ext4_ext_binsearch(inode, path, block);
 } else {
 // Index level - find next node
 idx = ext4_ext_binsearch_idx(inode, path, block);
 path[i+1].p_idx = idx;
 // Read next block
 path[i+1].p_bh = ext4_read_block(inode, ext4_idx_pblock(idx));
 path[i+1].p_hdr = (struct ext4_extent_header *) path[i+1].p_bh->b_data;
 }
 }
 return 0;
}

// Binary search in leaf level
static void ext4_ext_binsearch(struct inode *inode,
 struct ext4_ext_path *path, ext4_lblk_t block) {
 struct ext4_extent_header *eh = path->p_hdr;
 struct ext4_extent *ext = ext4_ext_idx_to_ext(eh);
 int low = 0, high = eh->eh_entries - 1;

 while (low <= high) {
 mid = (low + high) / 2;
 if (block < le32_to_cpu(ext[mid].ee_block))
 high = mid - 1;
 else if (block >= le32_to_cpu(ext[mid].ee_block) +
 le16_to_cpu(ext[mid].ee_len))
 low = mid + 1;
 else {
 // Found the extent
 path->p_ext = &ext[mid];
 return;
 }
 }
 // Not found - return position where it would be
 path->p_ext = (low > 0) ? &ext[low-1] : NULL;
}
```

Extent Tree Operations: Insert

Insert Algorithm

```
int ext4_ext_insert_extent(handle_t *handle, struct inode *inode,
 struct ext4_ext_path *path,
 struct ext4_extent *newext, int flag) {
 struct ext4_extent_header *eh;
 int depth = ext_depth(inode);
 int err;

 // Try to merge with existing extents
 if (ext4_ext_try_to_merge(inode, path, newext)) {
 // Merged successfully - no new extent needed
 return 0;
 }

 // Check if leaf has space
 eh = path[depth].p_hdr;
 if (eh->eh_entries < eh->eh_max) {
 // Insert in current leaf
 ext4_ext_insert_index(handle, inode, path, newext);
 } else {
 // Leaf is full - need to split
 err = ext4_ext_split(handle, inode, path, newext, flag);
 if (err) return err;
 }

 // Update extent cache
 ext4_ext_cache_insert(inode, newext);
 return 0;
}

// Extent merging logic
static int ext4_ext_try_to_merge(struct inode *inode,
 struct ext4_ext_path *path,
 struct ext4_extent *newext) {
 struct ext4_extent *ex;
 int depth = ext_depth(inode);

 // Try merging with left neighbor
 ex = path[depth].p_ext;
 if (ex > EXT_FIRST_EXTENT(path[depth].p_hdr)) {
 ex--;
 if (ext4_ext_can_merge(ex, newext)) {
 // Merge: extend left extent
 le16_add_cpu(&ex->ee_len, le16_to_cpu(newext->ee_len));
 return 1;
 }
 }

 // Try merging with right neighbor
 ex = path[depth].p_ext;
 if (ex < EXT_LAST_EXTENT(path[depth].p_hdr)) {
 ex++;
 if (ext4_ext_can_merge(newext, ex)) {
 // Merge: extend new extent into right
 le16_add_cpu(&newext->ee_len, le16_to_cpu(ex->ee_len));
 // Remove right extent
 ext4_ext_remove_space(inode, le32_to_cpu(ex->ee_block),
 le32_to_cpu(ex->ee_block) + le16_to_cpu(ex->ee_len) - 1);
 return 1;
 }
 }
 return 0;
}
```

Extent Tree Operations: Split

Node Splitting Algorithm

```
static int ext4_ext_split(handle_t *handle, struct inode *inode,
 struct ext4_ext_path *path,
 struct ext4_extent *newext, int flag) {
 int depth = ext_depth(inode);
 int err = 0;
 int i;

 // Allocate new blocks for split
 for (i = depth; i >= 0; i--) {
 if (i == depth) {
 // Split leaf node
 err = ext4_ext_split_leaf(handle, inode, path, newext);
 } else {
 // Split index node
 err = ext4_ext_split_index(handle, inode, path, i);
 }
 if (err) goto cleanup;
 }

 // Rebalance tree if needed
 if (ext_depth(inode) > 4) {
 // Tree too deep - try to compact
 ext4_ext_try_to_compact(inode);
 }
 return 0;
}

static int ext4_ext_split_leaf(handle_t *handle, struct inode *inode,
 struct ext4_ext_path *path,
 struct ext4_extent *newext) {
 struct ext4_extent_header *eh = path->p_hdr;
 struct ext4_extent *exts = EXT_FIRST_EXTENT(eh);
 int entries = eh->eh_entries;
 int split_point = entries / 2;

 // Allocate new leaf block
 new_block = ext4_new_meta_blocks(handle, inode, 0, NULL, &err);

 // Copy second half to new block
 memcpy(new_exts, &exts[split_point],
 (entries - split_point) * sizeof(struct ext4_extent));
 new_eh->eh_entries = entries - split_point;

 // Update current block (first half)
 eh->eh_entries = split_point;

 // Update parent index to point to new block
 ext4_ext_update_parent(handle, inode, path, new_block,
 le32_to_cpu(exts[split_point].ee_block));

 // Insert the new extent in appropriate leaf
 if (le32_to_cpu(newext->ee_block) < le32_to_cpu(exts[split_point].ee_block))
 // Insert in original leaf
 ext4_ext_insert_in_leaf(handle, inode, path, newext);
 else
 // Insert in new leaf
 ext4_ext_insert_in_leaf(handle, inode, new_path, newext);
 return 0;
}
```

Delayed Allocation (delalloc)

Core Concepts

Traditional (Ext3)

```
write()
  → allocate blocks
  → copy data
  → mark dirty
  → writeback
(Blocks allocated immediately)
```

Delayed Allocation (Ext4)

```
write()
  → reserve space
  → copy data
  → mark dirty
  → writeback
  → allocate blocks
(Allocation deferred to writeback time)
```

Benefits

Better allocation decisions

See full write pattern before allocating

Larger extents

Can allocate contiguous blocks spanning multiple writes

Reduced fragmentation

Fewer, larger extents

Lower CPU overhead

Fewer allocation calls

```
// Write begin with delalloc
static int ext4_da_write_begin(struct file *file, struct address_space *mapping,
 loff_t pos, unsigned len,
 struct page **pagep, void **fsdata) {
 struct inode *inode = mapping->host;
 // Reserve blocks (but don't allocate)
 ext4_da_reserve_space(inode, len);
 // Grab page
 page = grab_cache_page_write_begin(mapping, index);
 // Start journal transaction
 handle = ext4_journal_start(inode, EXT4_HT_WRITE_PAGE,
 ext4_da_write_credits(inode, len));
 return 0;
}

// Writeback time - allocate
static int ext4_da_writepages(struct address_space *mapping,
 struct writeback_control *wbc) {
 // Now allocate the reserved blocks
 while (pages_to_write) {
 ext4_map_blocks(handle, inode, map, EXT4_GET_BLOCKS_CREATE);
 io_submit(io_end);
 }
}
```

Multiblock Allocator (mballoc)

mballoc Architecture

Design Goals

- Allocate multiple blocks in one operation
- Reduce CPU overhead
- Improve locality
- Reduce fragmentation

Allocation Criteria Levels

```
// Search criteria (in order of preference)
#define CR_POWER2_ALIGNED  0  /* Aligned to power of 2 */
#define CR_GOAL_LEN_SLOW   1  /* Goal length with slower
search */
#define CR_BEST_AVAIL_LEN  2  /* Best available length */
#define CR_ANY_FREE        3  /* Any free block */
#define CR_MAX             4
```

Key Structures

```
struct ext4_allocation_context {
 struct inode *ac_inode; /* File being allocated */
 struct super_block *ac_sb; /* Superblock */
 ext4_lblk_t ac_l_ex; /* Logical extent start */
 ext4_fsblk_t ac_p_ex; /* Physical extent start */
 ext4_lblk_t ac_o_ex; /* Original extent length */
 unsigned int ac_g_ex; /* Goal extent length */
 int ac_criteria; /* Allocation criteria */
 struct ext4_prealloc_space *ac_pa; /* Preallocation space */
 struct ext4_locality_group *ac_lg; /* Locality group */
};

struct ext4_group_info {
 unsigned long bb_free; /* Free blocks */
 unsigned long bb_fragments; /* Fragments */
 unsigned int bb_largest_free_order; /* Largest free order */
 struct list_head bb_prealloc_list; /* Preallocated extents */
 void *bb_bitmap; /* Block bitmap */
 struct rw_semaphore alloc_sem; /* Allocation semaphore */
};
```

mballoc Allocation Flow

Allocation Algorithm

```
static int ext4_mb_regular_allocator(struct ext4_allocation_context *ac) {
 struct ext4_sb_info *sbi = EXT4_SB(ac->ac_sb);
 int err = 0;

 // Try each criteria level
 for (int cr = 0; cr < CR_MAX; cr++) {
 // Scan groups based on criteria
 ext4_mb_scan_groups(ac, cr);
 // Found allocation?
 if (ac->ac_status == AC_STATUS_FOUND) break;
 }

 if (ac->ac_status == AC_STATUS_FOUND) {
 // Allocate blocks
 err = ext4_mb_mark_used(ac);
 // Add to buddy cache
 ext4_mb_put_pa(ac, ac->ac_pa);
 } else {
 // No space - try to free some
 ext4_mb_discard_preallocations(ac->ac_sb);
 }
 return err;
}

// Buddy cache for free block tracking
static void ext4_mb_scan_groups(struct ext4_allocation_context *ac, int cr) {
 for (group = ac->ac_start; group < sbi->s_groups_count; group++) {
 grp = ext4_get_group_info(sb, group);

 // Check if group can satisfy request
 if (grp->bb_free < ac->ac_g_ex.fe_len) continue;

 // Check fragmentation
 if (grp->bb_fragments > ac->ac_g_ex.fe_len) continue;

 // Search within group
 ext4_mb_find_by_order(ac, grp, cr);
 if (ac->ac_status == AC_STATUS_FOUND) break;
 }
}
```

Preallocation Strategies

Inode Preallocation

```
struct ext4_prealloc_space {
    struct list_head  pa_inode_list;  /* List for inode */
    struct list_head  pa_group_list;  /* List for group */
    struct inode     *pa_inode;       /* Inode (for inode prealloc) */
    ext4_lblk_t       pa_lstart;      /* Logical start */
    ext4_fsblk_t      pa_pstart;      /* Physical start */
    ext4_lblk_t       pa_len;         /* Length */
    int               pa_type;        /* Prealloc type */
    spinlock_t        pa_lock;        /* Lock for updates */
    atomic_t          pa_count;       /* Reference count */
};

// Preallocation types
#define EXT4_MB_PO_INODE  0  /* Inode-specific preallocation */
#define EXT4_MB_PO_GROUP  1  /* Group-specific preallocation */
```

Locality Group Preallocation

```
// Per-CPU locality group
struct ext4_locality_group {
 struct list_head lg_prealloc_list; /* Preallocations for this CPU */
 spinlock_t lg_lock; /* Lock for list */
 struct ext4_prealloc_space *lg_prealloc; /* Active preallocation */
};

// Assign to locality group
static void ext4_mb_new_preallocation(struct ext4_allocation_context *ac) {
 if (S_ISREG(ac->ac_inode->i_mode)) {
 // Try to use inode preallocation
 ext4_mb_normalize_request(ac, &ac->ac_g_ex);
 // Fall back to locality group
 if (!ac->ac_pa) {
 ac->ac_lg = &EXT4_SB(sb)->s_locality_groups[get_cpu()];
 ext4_mb_get_locality_group(ac->ac_lg);
 }
 }
}
```

Fragmentation Handling

Types of Fragmentation

1. File Fragmentation

```
Ideal:
[File A] [File B] [File C]  ← Contiguous

Fragmented:
[A1][B1][A2][C1][B2][A3][C2] ← Interleaved
```

2. Free Space Fragmentation

```
Ideal:
[Free] [File A] [Free] [File B] ← Large free holes

Fragmented:
[Free][File][Free][File][Free]  ← Many small holes
```

Fragmentation Metrics

```
// Kernel fragment count
void ext4_mb_measure_fragmentation(struct super_block *sb) {
    for (group = 0; group < ngroups; group++) {
        grp = ext4_get_group_info(sb, group);
        fragments = ext4_mb_count_fragments(grp);
        if (fragments > sbi->s_mb_max_fragments) {
            // Too fragmented - trigger defrag
            ext4_mb_defrag_group(sb, group);
        }
    }
}
```

Defragmentation Tools

```
# Online defragmentation (file-level)
e4defrag -v /path/to/file

# Directory/entire filesystem
e4defrag /mount/point

# Check fragmentation level
filefrag -v /path/to/file
# Output: 10 extents found, perfection would be 1 extent

# Kernel fragmentation info
cat /sys/fs/ext4/sda1/fragments_count
cat /sys/fs/ext4/sda1/extents_count
```

Persistent Fragmentation Strategies

Prevention Over Cure

1

Delayed Allocation

```
// Wait for more data before allocating
static int ext4_da_writepages(...) {
    while (pages_to_write > MIN_PAGES_TO_ALLOC) {
        map->m_len = min(pages_to_write, MAX_EXTENT_LEN);
        ext4_map_blocks(handle, inode, map,
                        EXT4_GET_BLOCKS_CREATE);
    }
}
```

2

Preallocation Hints

```
// Persistent preallocation (fallocate)
int ext4_fallocate(struct file *file, int mode,
                   loff_t offset, loff_t len) {
    ext4_ext_map_blocks(handle, inode, map,
        EXT4_GET_BLOCKS_PRE_IO | EXT4_GET_BLOCKS_CREATE);
}
// User-space hints
posix_fallocate(fd, offset, len);
fallocate(fd, FALLOC_FL_KEEP_SIZE, offset, len);
```

3

Stride and Strip Width

```
# For RAID arrays
mkfs.ext4 -E stride=64,stripe_width=128 /dev/md0
# Mount with alignment hints
mount -o stripe=64 /dev/md0 /mnt
```

4

Reserved Blocks

```
# Reserve blocks for root (5% default)
tune2fs -m 2 /dev/sda1 # Reduce to 2%
# Prevent ENOSPC fragmentation
echo 10 > /proc/sys/fs/ext4/mb_min_to_scan
```

Low-Memory Handling

Memory Pressure Responses

Shrinker Callback

```
// Register with memory management
static struct shrinker ext4_es_shrinker = {
    .count_objects = ext4_es_count_objects,
    .scan_objects  = ext4_es_scan_objects,
    .seeks         = DEFAULT_SEEKS,
};

// Called under memory pressure
static unsigned long ext4_es_scan_objects(
        struct shrinker *shrink,
        struct shrink_control *sc) {
    struct ext4_sb_info *sbi =
        container_of(shrink, struct ext4_sb_info,
                     s_es_shrinker);
    // Scan extent status cache
    nr = sc->nr_to_scan ?
        percpu_counter_read_positive(
            &sbi->s_es_stats.es_stats_shk_cnt) : 0;
    // Free entries under pressure
    while (nr && ext4_es_free_extent(sbi, 1)) {
        nr--;
    }
    return percpu_counter_read_positive(
        &sbi->s_es_stats.es_stats_shk_cnt);
}
```

Page Cache Pruning

```
// Under memory pressure
static void ext4_free_dirty_pages(struct super_block *sb) {
 struct address_space *mapping =
 sb->s_bdev->bd_inode->i_mapping;

 // Write back dirty pages
 writeback_inodes_sb(sb, WB_REASON_VMSCAN);

 // Invalidate clean pages
 invalidate_inode_pages2(mapping);

 // Reclaim inodes
 prune_icache_sb(sb, 100);
}
```

ENOSPC Handling

Out-of-Space Recovery Paths

ENOSPC Detection

```
int ext4_has_free_blocks(
        struct ext4_sb_info *sbi,
                    s64 nblocks) {
    s64 free_blocks, dirty_blocks, root_blocks;

    free_blocks  = percpu_counter_read_positive(
            &sbi->s_freeblocks_counter);
    dirty_blocks = percpu_counter_read_positive(
            &sbi->s_dirtyclusters_counter);

    // Account for reserved blocks
    if (!capable(CAP_SYS_RESOURCE))
        root_blocks =
            ext4_r_blocks_count(sbi->s_es);

    // Available = free - dirty - reserved
    return (free_blocks - dirty_blocks
          - root_blocks) >= nblocks;
}
```

Recovery Strategies

```
// When ENOSPC occurs
static int ext4_da_writepages(struct address_space *mapping,
 struct writeback_control *wbc) {
 // 1. Try to free space
 ext4_mb_discard_preallocations(inode->i_sb);

 // 2. Flush journal to free blocks
 jbd2_journal_force_commit(journal);

 // 3. Drop unused inodes
 shrink_icache_memory(1000, GFP_KERNEL);

 // 4. If still no space, fall back to non-delalloc
 if (!ext4_has_free_blocks(sbi, needed)) {
 // Write pages without delalloc
 return ext4_writepages(mapping, wbc);
 }

 // Continue with delalloc
 return ext4_da_write_pages(mapping, wbc);
}
```

Orphan Inode Cleanup

Orphan Inode Management

What are Orphan Inodes? Inodes that were unlinked but still have open file handles.

Orphan List

```
// Inode flagged as orphan
static int ext4_orphan_add(handle_t *handle,
                           struct inode *inode) {
    struct ext4_sb_info *sbi = EXT4_SB(inode->i_sb);

    // Add to in-memory list
    list_add(&EXT4_I(inode)->i_orphan, &sbi->s_orphan);

    // Update on-disk orphan list
    if (!ext4_test_inode_state(inode,
                               EXT4_STATE_ORPHAN_FILE)) {
        ext4_orphan_file_add(handle, inode);
    }
}

// Cleanup on last close
void ext4_orphan_del(handle_t *handle, struct inode *inode) {
    // Remove from list
    list_del_init(&EXT4_I(inode)->i_orphan);
    // Update on-disk
    ext4_orphan_file_del(handle, inode);
}
```

Recovery on Mount

```
// During mount, clean up orphans
static void ext4_orphan_cleanup(struct super_block *sb,
 struct ext4_super_block *es) {
 struct inode *inode;

 // Walk orphan list
 for (ino = ext4_orphan_file_first(sb);
 ino;
 ino = ext4_orphan_file_next(sb, ino)) {
 inode = ext4_iget(sb, ino);
 if (inode && inode->i_nlink) {
 // Link count > 0 - partial delete
 ext4_truncate(inode);
 } else {
 // No links - delete completely
 ext4_free_inode(inode);
 }
 iput(inode);
 }
}
```

Module 5 Summary: Extent-Based Allocation Key Takeaways

Extent Tree Operations

- **Lookup:** O(log n) binary search
- **Insert:** Merge when possible, split when full
- **Delete:** Coalesce adjacent extents
- **Split:** 50/50 split at node level

Delayed Allocation

- Allocate at writeback, not write time
- Better contiguous allocation
- Risk: ENOSPC on writeback
- Critical for SSD/Flash performance

mballoc

- Allocates multiple blocks at once
- Preallocation strategies
- Buddy cache for free block tracking
- Locality groups per CPU

Fragmentation

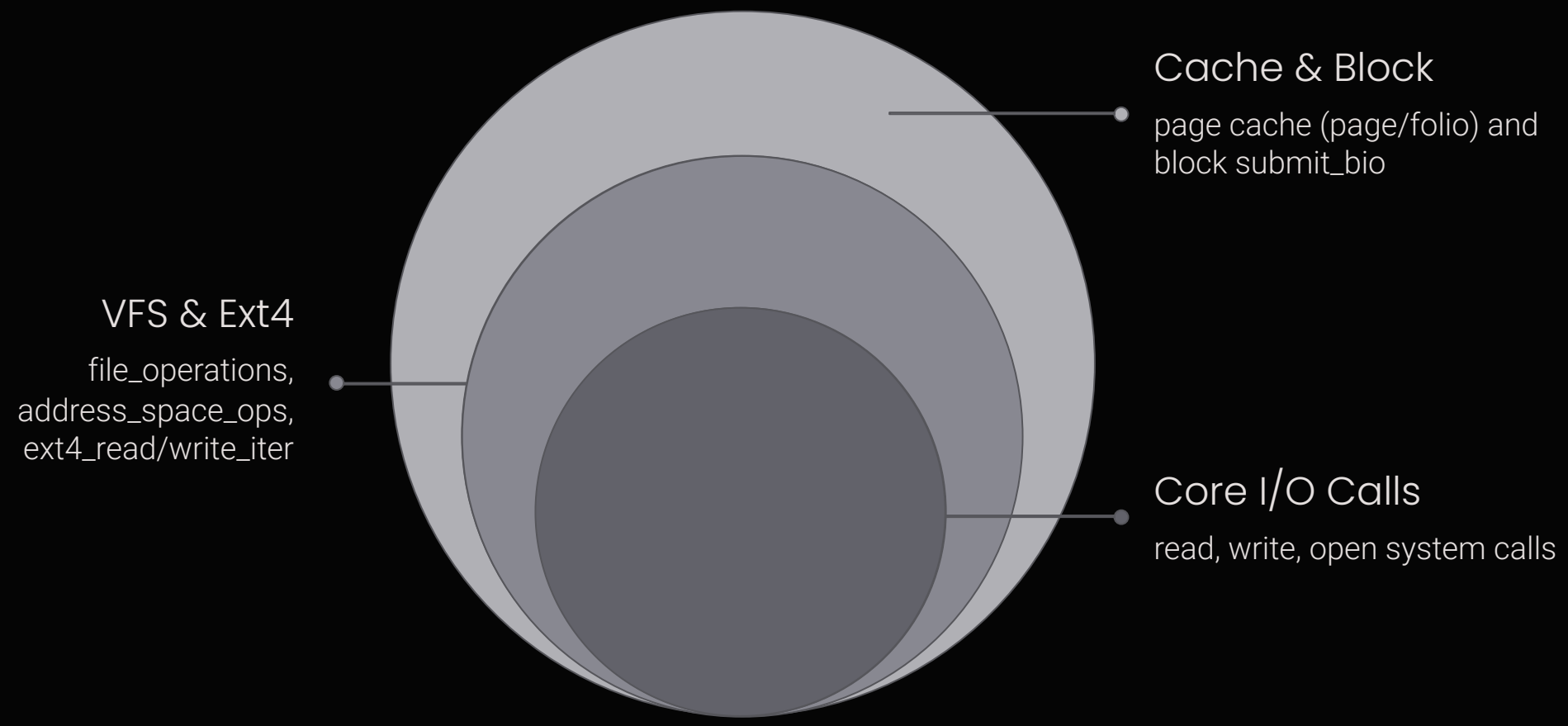
- Prevent with delalloc and preallocation
- Monitor with filefrag
- Fix with e4defrag
- Tune with stride/stripe

Low-Memory/ENOSPC

- Shrinker callbacks free cache
- Orphan cleanup on mount
- Multiple recovery strategies
- Fallback to non-delalloc

Module 6: I/O Path & Memory Integration

VFS Integration Overview — Architecture Stack



Each layer in the stack has a well-defined interface, enabling clean separation of concerns from user-space system calls all the way down to physical block I/O submission.

file_operations Structure

Ext4 File Operations

```
const struct file_operations ext4_file_operations = {
 .llseek = ext4_llseek,
 .read_iter = ext4_file_read_iter, // AIO-aware read
 .write_iter = ext4_file_write_iter, // AIO-aware write
 .mmap = ext4_file_mmap,
 .open = ext4_file_open,
 .flush = ext4_file_flush,
 .release = ext4_file_release,
 .fsync = ext4_sync_file,
 .fallocate = ext4_fallocate,
 .splice_read = generic_file_splice_read,
 .splice_write = iter_file_splice_write,
 .unlocked_ioctl = ext4_ioctl,
 .compat_ioctl = ext4_compat_ioctl,
};

// Directory operations
const struct file_operations ext4_dir_operations = {
 .llseek = ext4_dir_llseek,
 .read = generic_read_dir,
 .iterate_shared = ext4_readdir,
 .fsync = ext4_sync_file,
 .unlocked_ioctl = ext4_ioctl,
};
```

address_space_operations

Page Cache Integration

```
const struct address_space_operations ext4_aops = {
 .read_folio = ext4_read_folio, // Read a single page
 .readahead = ext4_readahead, // Prefetch pages
 .writepages = ext4_writepages, // Write multiple pages
 .write_begin = ext4_write_begin, // Start write to page
 .write_end = ext4_write_end, // End write to page
 .bmap = ext4_bmap, // Block mapping
 .invalidate_folio = ext4_invalidate_folio, // Invalidate page
 .release_folio = ext4_release_folio, // Release page
 .direct_IO = ext4_direct_IO, // Direct I/O
 .migrate_folio = buffer_migrate_folio, // Page migration
 .is_partially_uptodate = block_is_partially_uptodate,
 .error_remove_page = generic_error_remove_page,
 .swap_activate = ext4_swap_activate, // Swap file support
 .swap_deactivate = ext4_swap_deactivate,
};

// Extents-specific operations (for files with extents)
const struct address_space_operations ext4_da_aops = {
 .read_folio = ext4_read_folio,
 .readahead = ext4_readahead,
 .writepages = ext4_da_writepages, // Delalloc aware
 .write_begin = ext4_da_write_begin, // Delalloc write
 .write_end = ext4_da_write_end, // Delalloc complete
 .bmap = ext4_bmap,
 .invalidate_folio = ext4_invalidate_folio,
 .release_folio = ext4_release_folio,
 .direct_IO = ext4_direct_IO,
 .migrate_folio = buffer_migrate_folio,
};
```

Read Path: ext4_read_iter()

Read Flow Deep Dive

```
// Entry point for reads
ssize_t ext4_file_read_iter(struct kiocb *iocb, struct iov_iter *to) {
 struct inode *inode = file_inode(iocb->ki_filp);

 // Handle encrypted files
 if (IS_ENCRYPTED(inode) && S_ISREG(inode->i_mode)) {
 return ext4_encrypted_read_iter(iocb, to);
 }
 // Use generic read_iter
 return generic_file_read_iter(iocb, to);
}

// Generic read implementation
ssize_t generic_file_read_iter(struct kiocb *iocb, struct iov_iter *iter) {
 if (iocb->ki_flags & IOCB_DIRECT) {
 // Direct I/O path
 return generic_file_direct_read(iocb, iter);
 }
 // Buffered I/O path
 return generic_file_buffered_read(iocb, iter, ret);
}

// Buffered read with page cache
static ssize_t generic_file_buffered_read(struct kiocb *iocb,
 struct iov_iter *iter,
 ssize_t written) {
 struct file *filp = iocb->ki_filp;
 struct address_space *mapping = filp->f_mapping;
 struct page *page;

 for (;;) {
 // Find page in cache
 page = find_get_page(mapping, index);
 if (!page) {
 // Not in cache - read from disk
 page = page_cache_sync_readahead(mapping, ra, filp,
 index, last_index - index);
 page = find_get_page(mapping, index);
 }
 // Copy data to userspace
 copy_page_to_iter(page, offset, bytes, iter);
 written += bytes;
 // Move to next page
 index++;
 }
 return written;
}
```

Write Path: ext4_write_iter()

Write Flow Deep Dive

```
// Entry point for writes
ssize_t ext4_file_write_iter(struct kiocb *iocb, struct iov_iter *from) {
 struct inode *inode = file_inode(iocb->ki_filp);

 // Check for O_APPEND
 if (iocb->ki_flags & IOCB_APPEND) {
 iocb->ki_pos = i_size_read(inode);
 }

 // Handle direct I/O
 if (iocb->ki_flags & IOCB_DIRECT) {
 return ext4_direct_IO_write(iocb, from);
 }

 // Buffered write
 return ext4_buffered_write_iter(iocb, from);
}

// Buffered write implementation
ssize_t ext4_buffered_write_iter(struct kiocb *iocb, struct iov_iter *from) {
 struct file *file = iocb->ki_filp;
 struct inode *inode = file->f_mapping->host;
 ssize_t ret;

 // Reserve quota
 ret = dquot_reserve_space(inode, bytes);
 if (ret) return ret;

 // Start journal transaction
 handle = ext4_journal_start(inode, EXT4_HT_WRITE_PAGE, credits);

 // Use generic write_iter
 ret = generic_perform_write(file, from, iocb->ki_pos);

 // Update inode size if needed
 if (iocb->ki_pos + ret > i_size_read(inode)) {
 ext4_update_inode_size(inode, iocb->ki_pos + ret);
 }

 // End journal transaction
 ext4_journal_stop(handle);
 return ret;
}
```

Page Cache and Writeback

Writeback Mechanisms

Dirty Page Tracking

```
// Page marked dirty
void set_page_dirty(struct page *page) {
    struct address_space *mapping = page->mapping;

    // Add to dirty list
    if (!TestSetPageDirty(page)) {
        spin_lock(&mapping->private_lock);
        list_add(&page->lru, &mapping->dirty_pages);
        spin_unlock(&mapping->private_lock);
        // Notify filesystem
        mapping->a_ops->set_page_dirty(page);
    }
}
```

Writeback Triggers

```
// Background writeback
void wakeup_flusher_threads(enum wb_reason reason) {
 // Start background writeback
 bdi_writeback_workfn(bdi);
}

// Sync writeback
int sync_inodes_sb(struct super_block *sb,
 struct writeback_control *wbc) {
 // Write all dirty inodes
 writeback_inodes_sb(sb, wbc);
}

// Per-inode writeback
int ext4_writepages(struct address_space *mapping,
 struct writeback_control *wbc) {
 // Collect dirty pages for this inode
 while (index <= end) {
 page = find_get_page(mapping, index);
 if (page && PageDirty(page)) {
 // Write page
 ext4_writepage(page, wbc);
 }
 }
}
```

Direct I/O vs Buffered I/O

Comparison

Buffered I/O (default)

Write Path:
Userspace → Page Cache → Writeback → Disk

Benefits:

- Fast writes (async)
- Read caching
- Merge operations

Drawbacks:

- Data loss window
- Extra copy
- Writeback latency

Direct I/O

Write Path:
Userspace → Disk (bypass cache)

Benefits:

- No double caching
- Application-controlled
- No data loss window

Drawbacks:

- Must be sector-aligned
- No read caching
- Potential fragmentation

Implementation

```
// Direct I/O write
ssize_t ext4_direct_IO_write(struct kiocb *iocb,
 struct iov_iter *iter) {
 struct inode *inode =
 iocb->ki_filp->f_mapping->host;
 loff_t pos = iocb->ki_pos;

 // Check alignment
 if (!IS_ALIGNED(pos | iov_iter_alignment(iter),
 i_blocksize(inode))) {
 return -EINVAL;
 }

 // Bypass page cache
 ret = __blockdev_direct_IO(iocb, inode, iter,
 ext4_get_block_dio,
 NULL, 0);

 // Flush journal if needed
 if (ret > 0 && (iocb->ki_flags & IOCB_DSYNC)) {
 ext4_sync_file(iocb->ki_filp, 0, LLONG_MAX, 1);
 }
 return ret;
}
```

Read-Ahead Mechanism

Predictive Read Caching

Read-Ahead Algorithm

```
void ext4_readahead(struct readahead_control *rac) {
    struct inode *inode = rac->mapping->host;
    struct ext4_map_blocks map;
    struct page *page;

    // Calculate read-ahead window
    start    = readahead_index(rac);
    nr_pages = readahead_count(rac);

    // Get extent mapping
    map.m_lblk = start;
    map.m_len  = nr_pages;
    ext4_map_blocks(NULL, inode, &map, 0);

    // Submit I/O for the range
    for (index = start; index < start + nr_pages; index++) {
        page = readahead_page(rac);
        if (map.m_pblk && index >= map.m_lblk &&
            index < map.m_lblk + map.m_len) {
            // Page is in extent - submit I/O
            submit_page_read(page,
                map.m_pblk + (index - map.m_lblk));
        } else {
            // Page not in extent - skip
            unlock_page(page);
            put_page(page);
        }
    }
}
```

Adaptive Read-Ahead

```
struct file_ra_state {
    pgoff_t     start;       /* Where readahead started */
    unsigned int size;       /* # of readahead pages */
    unsigned int async_size; /* Do async readahead */
    unsigned int ra_pages;   /* Maximum readahead window */
    int         mmap_miss;   /* Cache miss count for mmap */
    loff_t      prev_pos;    /* Previous read position */
};
```

Kernel Tunables

```
# 256KB read-ahead
echo 256 > /sys/block/sda/queue/read_ahead_kb
```

AIO and io_uring Integration

Async I/O Support

Traditional AIO

```
// AIO submission
int ext4_file_read_iter(struct kiocb *iocb,
                        struct iov_iter *to) {
    // Check for async
    if (iocb->ki_flags & IOCB_HIPRI) {
        // High priority I/O
        return ext4_direct_IO_async(iocb, to);
    }
    // Fallback to sync
    return generic_file_read_iter(iocb, to);
}
```

io_uring Support

```
// io_uring integration
static int ext4_uring_cmd(struct io_uring_cmd *cmd,
                          unsigned int issue_flags) {
    struct inode *inode = file_inode(cmd->file);
    struct ext4_io_uring *iou;

    // Handle io_uring commands
    switch (cmd->cmd_op) {
    case EXT4_URING_CMD_TRIM:
        // Batch discard operations
        return ext4_trim_fs(inode->i_sb, cmd);
    case EXT4_URING_CMD_FALLOCATE:
        // Async fallocate
        return ext4_fallocate_async(inode, cmd);
    }
}
```

io_uring Performance

- Lower latency than AIO
- Batched submission
- Fixed buffers for zero-copy

Folio Interface

Modern Memory Management

What are Folios?

```
// Traditional page (4KB)
struct page {
    unsigned long       flags;
    struct address_space *mapping;
    pgoff_t             index;
    atomic_t            _refcount;
    // ... other fields
};

// Folio (can be compound of multiple pages)
struct folio {
    struct page  page;          // First page
    unsigned long private;      // Filesystem private data
    spinlock_t   _page_1_lock;  // Lock for first page
    // ... compound page data
};
```

Folio Operations

```
// Convert page to folio
static inline struct folio *page_folio(struct page *page) {
 return (struct folio *)page;
}

// Folio-aware filesystem operations
int ext4_read_folio(struct file *file, struct folio *folio) {
 struct inode *inode = folio->mapping->host;
 // Handle large folios (>4KB)
 if (folio_size(folio) > PAGE_SIZE) {
 return ext4_read_large_folio(inode, folio);
 }
 // Traditional read
 return block_read_full_folio(folio, ext4_get_block);
}

// Folio invalidation
void ext4_invalidate_folio(struct folio *folio,
 size_t offset, size_t length) {
 // Invalidate part of a large folio
 if (folio_size(folio) > PAGE_SIZE) {
 ext4_invalidate_large_folio(folio, offset, length);
 } else {
 block_invalidate_folio(folio, offset, length);
 }
}
```

Page Locks and Memory Management

Locking Mechanisms

Page Lock Types

```
// PG_locked    - I/O in progress
// PG_writeback - Writeback in progress
// PG_dirty     - Modified, needs writeback
// PG_uptodate  - Page contains valid data

// Lock ordering
void ext4_page_locking_order(void) {
    // 1. Inode lock (i_rwsem)
    // 2. Page lock (lock_page)
    // 3. Buffer head lock (bh->b_lock)
    // 4. Journal handle
}

// Stuck page debugging
static void debug_stuck_page(struct page *page) {
    printk(KERN_ERR "Page stuck: index=%lu, flags=%lx\n",
           page->index, page->flags);
    printk(KERN_ERR "Mapping: %ps\n",
           page->mapping->a_ops);
    printk(KERN_ERR "Refcount: %d\n",
           page_ref_count(page));
    dump_stack();
}
```

Low-Memory Page Reclaim

```
// Under memory pressure
static int ext4_release_folio(struct folio *folio,
 gfp_t wait) {
 struct inode *inode = folio->mapping->host;

 // Check if page can be released
 if (folio_test_writeback(folio))
 return 0; // Still writing

 if (folio_test_dirty(folio))
 return 0; // Still dirty

 // Release buffer heads
 try_to_free_buffers(folio);
 return 1;
}
```

Zoned Device Support

Zoned Block Devices (ZBD)

Zone Types

```
Conventional Zones: Random write capable
┌──────────────────────────────────────────────────────────────────────────────┐
│ Write anywhere (like regular disk)                                           │
└──────────────────────────────────────────────────────────────────────────────┘

Sequential Zones: Write pointer only
┌──────────────────────────────────────────────────────────────────────────────┐
│ Write → Write pointer advances                                               │
│ Cannot overwrite without reset                                               │
└──────────────────────────────────────────────────────────────────────────────┘
```

Ext4 Zone Awareness

```
// Check if device is zoned
static bool ext4_dev_is_zoned(struct super_block *sb) {
 if (bdev_is_zoned(sb->s_bdev))
 return true;
 return false;
}

// Zone-aware allocation
static int ext4_zone_alloc(struct inode *inode,
 ext4_lblk_t lblk,
 ext4_fsblk_t *pblk,
 unsigned int len) {
 struct super_block *sb = inode->i_sb;
 struct blk_zone zone;

 // Get current write pointer
 ret = blkdev_zone_mgmt(sb->s_bdev,
 REQ_OP_ZONE_REPORT,
 start_sector, &zone);

 // Allocate from write pointer position
 if (zone.type == BLK_ZONE_TYPE_SEQWRITE_REQ) {
 *pblk = zone.wp_sector >>
 (sb->s_blocksize_bits - 9);
 }
 return 0;
}
```

4KB Block vs 16KB Page Alignment

Alignment Challenges

Block vs Page Size Mismatch

```
4KB Block Size with 16KB Page:

┌─────────────────────────────────────────────────────────────┐
│          16KB Page                                          │
├──────────────┬──────────────┬──────────────┬──────────────┤
│ Block 0      │ Block 1      │ Block 2      │ Block 3      │
├──────────────┼──────────────┼──────────────┼──────────────┤
│ Data A       │ Data B       │ Data C       │ Data D       │
└──────────────┴──────────────┴──────────────┴──────────────┘

Issue: Partial page writes require read-modify-write
```

buffer_head Interaction

```
// Buffer head for each block
struct buffer_head {
 struct buffer_head *b_this_page; /* Circular list */
 struct page *b_page; /* Associated page */
 sector_t b_blocknr; /* Block number */
 size_t b_size; /* Block size */
 char *b_data; /* Data pointer */
 unsigned long b_state; /* Buffer state */
};

// Map block to page
static int ext4_block_to_page(struct inode *inode, sector_t block) {
 // Calculate page index
 pgoff_t index = block >> (PAGE_SHIFT - inode->i_blkbits);

 // Get page
 struct page *page = find_or_create_page(mapping, index, GFP_NOFS);

 // Map buffer heads
 if (!page_has_buffers(page)) {
 create_empty_buffers(page, i_blocksize(inode), 0);
 }
 return page;
}
```

Hands-On Lab 1: I/O Tracing

blktrace Analysis

Setup

```
# Install tools
apt-get install blktrace fio

# Create test filesystem
dd if=/dev/zero of=/tmp/ext4_test.img bs=1M count=1024
mkfs.ext4 -F /tmp/ext4_test.img
mount -o loop /tmp/ext4_test.img /mnt/test

# Start tracing
blktrace -d /dev/loop0 -o trace -w 30
```

Trace Workloads

```
# Terminal 1: Run workload
cd /mnt/test
fio --name=seqwrite --rw=write --bs=4k \
                --size=256M
fio --name=randwrite --rw=randwrite --bs=4k \
                --size=256M
fio --name=seqread --rw=read --bs=4k  \
                --size=256M

# Terminal 2: Analyze trace
blkparse -i trace -o trace.out
btt -i trace -o analysis/
```

bpftrace I/O Tracing

```
# Trace ext4 read/write calls
cat > ext4_trace.bt << EOF
kprobe:ext4_file_read_iter {
 printf("read: %s size=%d\n",
 str(arg1->f_path.dentry->d_name.name),
 arg2->count);
}
kprobe:ext4_file_write_iter {
 printf("write: %s size=%d\n",
 str(arg1->f_path.dentry->d_name.name),
 arg2->count);
}
kprobe:ext4_writepages {
 printf("writeback: %s\n",
 str(((struct address_space *)arg0)
 ->host->i_sb->s_id));
}
EOF
bpftrace ext4_trace.bt
```

Hands-On Lab 2: Buffered vs Direct I/O

Performance Comparison — Setup

```
# Create test files
cd /mnt/test
fallocate -l 1G testfile

# Clear caches for accurate tests
echo 3 > /proc/sys/vm/drop_caches
```

Buffered I/O Test

```
# Sequential write with buffered I/O
fio --name=buffered-write \
    --filename=testfile \
    --rw=write \
    --bs=1M \
    --size=512M \
    --direct=0 \
    --fsync=0 \
    --output=buffered-write.json \
    --output-format=json

# Sequential read (warmed cache)
fio --name=buffered-read-warm \
    --filename=testfile \
    --rw=read \
    --bs=1M \
    --size=512M \
    --direct=0 \
    --output=buffered-read-warm.json

# Cold cache read
echo 3 > /proc/sys/vm/drop_caches
fio --name=buffered-read-cold \
    --filename=testfile \
    --rw=read \
    --bs=1M \
    --size=512M \
    --direct=0 \
    --output=buffered-read-cold.json
```

Direct I/O Test

```
# Direct I/O write
fio --name=direct-write \
    --filename=testfile \
    --rw=write \
    --bs=1M \
    --size=512M \
    --direct=1 \
    --fsync=0 \
    --output=direct-write.json

# Direct I/O read
echo 3 > /proc/sys/vm/drop_caches
fio --name=direct-read \
    --filename=testfile \
    --rw=read \
    --bs=1M \
    --size=512M \
    --direct=1 \
    --output=direct-read.json
```

Analysis Script

```
#!/bin/bash
echo "=== Buffered Write ==="
jq '.jobs[0].write.bw' buffered-write.json
jq '.jobs[0].write.iops' buffered-write.json
echo "=== Direct Write ==="
jq '.jobs[0].write.bw' direct-write.json
jq '.jobs[0].write.iops' direct-write.json
echo "=== Buffered Read (Cold) ==="
jq '.jobs[0].read.bw' buffered-read-cold.json
echo "=== Buffered Read (Warm) ==="
jq '.jobs[0].read.bw' buffered-read-warm.json
echo "=== Direct Read ==="
jq '.jobs[0].read.bw' direct-read.json
```

Hands-On Lab 3: Page Cache Analysis

Memory Pressure Experiments

Setup

```
# Create large test file
dd if=/dev/zero of=/mnt/test/largefile bs=1M count=2048

# Monitor memory
watch -n 1 'cat /proc/meminfo | \
  grep -E "^(Cached|Dirty|Writeback)"'
```

Page Cache Behaviour

```
# 1. Fill page cache
cat /mnt/test/largefile > /dev/null
# Observe Cached memory increase

# 2. Modify file to create dirty pages
dd if=/dev/zero of=/mnt/test/largefile \
   bs=1M count=1024 conv=notrunc
# Observe Dirty memory increase

# 3. Trigger writeback
sync
# Observe Dirty decrease, Writeback may spike

# 4. Drop caches
echo 1 > /proc/sys/vm/drop_caches  # Page cache
echo 2 > /proc/sys/vm/drop_caches  # Slab objects
echo 3 > /proc/sys/vm/drop_caches  # Both
```

Memory Pressure Simulation

```
# Use memory cgroup to limit available memory
mkdir /sys/fs/cgroup/memory/ext4_test
echo 100M > /sys/fs/cgroup/memory/ext4_test/\
 memory.limit_in_bytes

# Run workload under memory pressure
echo $$ > /sys/fs/cgroup/memory/ext4_test/tasks
fio --name=mem-pressure \
 --rw=write \
 --bs=4k \
 --size=200M \
 --filename=/mnt/test/pressure

# Observe reclaim behaviour
cat /sys/fs/cgroup/memory/ext4_test/memory.stat
```

Performance Profiling with perf

Kernel and Userspace Profiling

Trace Ext4 Functions

```
# Record ext4 events
perf record -e ext4:* -a -- sleep 10

# View report
perf report

# Sample ext4 function calls
perf record -e 'ext4:*' -a -g -- sleep 10
perf script | grep -E "(ext4_write|ext4_read)"

# Specific tracepoints
perf stat \
  -e ext4:ext4_es_lookup_extent_enter,\
     ext4:ext4_es_lookup_extent_exit \
  -a -- sleep 10
```

I/O Latency Analysis

```
# Trace block I/O latency
perf record \
  -e block:block_rq_issue,block:block_rq_complete \
  -a -- sleep 10

# Calculate latency
perf script | awk '
  /block_rq_issue/   { start[$4]=$2 }
  /block_rq_complete/ {
    if(start[$4])
      print $4, ($2-start[$4])*1000
  }'
```

Flame Graph Generation

```
# Record stack traces
perf record -F 99 -a -g -- sleep 30

# Generate flame graph
perf script \
 | stackcollapse-perf.pl \
 | flamegraph.pl > ext4_flame.svg

# Focus on ext4
perf report \
 --sort comm,dso,symbol \
 --graph=0.5
```

Module 6 Summary: I/O Path Key Takeaways

VFS Integration

- **file_operations:** System call handlers
- **address_space_ops:** Page cache operations
- **read_iter/write_iter:** AIO-aware I/O

I/O Modes

- **Buffered I/O:** Page cache, async writeback
- **Direct I/O:** Bypass cache, application control
- **AIO/io_uring:** Async I/O, reduced syscall overhead

Memory Management

- **Folios:** Compound pages for larger allocations
- **Page locks:** PG_locked, PG_writeback, PG_dirty
- **Reclaim:** Shrinker callbacks under pressure

Advanced Features

- **Read-ahead:** Predictive caching
- **Zoned devices:** Write pointer constraints
- **Alignment:** 4KB block vs 16KB page

Debugging Tools

- **blktrace:** Block I/O tracing
- **perf:** Kernel profiling
- **bpftrace:** Dynamic tracing
- **fio:** Workload generation

Q&A and Discussion

Open Topics

Discussion Questions

1. **Delayed allocation trade-offs**

   When would you disable it?

2. **Direct I/O vs Buffered**

   Which workloads benefit from each?

3. **Extent tree depth**

   When do you hit 5 levels?

4. **io_uring vs AIO**

   When to choose one over the other?

Common Issues

- **ENOSPC with delalloc:** Space reserved but not allocated
- **Stuck pages:** I/O deadlock scenarios
- **Fragmentation:** Degrading over time
- **Zoned device constraints:** Write pointer management

Best Practices

- Use fallocate for large files
- Monitor fragmentation with filefrag
- Tune read-ahead for sequential workloads
- Use fstrim regularly for SSD
- Consider noatime for high-performance systems

Preview of Day 4: Coming Up Tomorrow

1

Module 7: Advanced Ext4 Features

- Bigalloc, metadata checksumming
- Encryption (FBE/FDE) internals
- Android/Yocto extensions
- Garbage collection and TRIM

2

Module 8: Performance Analysis & Tuning

- Mount option optimisation
- Storage-specific tuning (eMMC, UFS, NVMe, HDD)
- Workload-specific strategies
- Memory pressure tuning

3

Hands-On Labs

- Encryption setup and debugging
- Mount flag benchmarking
- TRIM performance analysis

4

Preparation

- Review Day 3 concepts
- Prepare test devices (SSD, NVMe if available)
- Install encryption tools

Additional Resources

References and Further Reading

Kernel Documentation

- Documentation/filesystems/ext4/ext4.rst
- Documentation/block/stat.rst
- Documentation/admin-guide/blockdev/

Source Files to Study

- `fs/ext4/extents.c` — Extent tree operations
- `fs/ext4/mballoc.c` — Multiblock allocator
- `fs/ext4/inode.c` — I/O operations
- `fs/ext4/file.c` — File operations
- `fs/ext4/page-io.c` — Page I/O handling
- `mm/filemap.c` — Page cache core

Tools Reference

```
# I/O tracing
man blktrace
man blkparse
man btt

# Performance analysis
man perf
man bpftrace
man fio

# Debugging
echo "file fs/ext4/* +p" > \
 /sys/kernel/debug/dynamic_debug/control
cat /proc/fs/ext4/sda1/options
```

Deep Dive: Extent Tree Lookup — Binary Search Detail

The binary search at the leaf level is the core of O(log n) performance. Understanding the boundary conditions is critical for debugging and kernel development.

```
// Binary search in leaf level — boundary conditions
static void ext4_ext_binsearch(struct inode *inode,
                               struct ext4_ext_path *path,
                               ext4_lblk_t block) {
    struct ext4_extent_header *eh = path->p_hdr;
    struct ext4_extent *ext = ext4_ext_idx_to_ext(eh);
    int low = 0, high = eh->eh_entries - 1;

    while (low <= high) {
        mid = (low + high) / 2;
        if (block < le32_to_cpu(ext[mid].ee_block))
            high = mid - 1;
        else if (block >= le32_to_cpu(ext[mid].ee_block) +
                     le16_to_cpu(ext[mid].ee_len))
            low = mid + 1;
        else {
            // Found the extent
            path->p_ext = &ext[mid];
            return;
        }
    }
    // Not found - return position where it would be
    path->p_ext = (low > 0) ? &ext[low-1] : NULL;
}
```

Found Case

block falls within `ee_block` to `ee_block + ee_len - 1`

Not Found Case

Returns the extent just before the logical block position, or NULL if before all extents

Complexity

O(log n) per level × depth levels = O(depth × log entries)

Extent Tree: Merge Conditions

Two extents can be merged only when they are physically and logically adjacent. The `ext4_ext_can_merge()` function enforces these constraints.

Merge Eligibility Criteria

→ Logical Adjacency

The logical end of the left extent must equal the logical start of the right extent

→ Physical Adjacency

The physical end of the left extent must equal the physical start of the right extent

→ Length Constraint

Combined length must not exceed 32768 blocks (the maximum `ee_len` value)

→ Initialised Flag

Both extents must have the same initialised/unwritten state

Why Merging Matters

Merging reduces the number of extents in the tree, which:

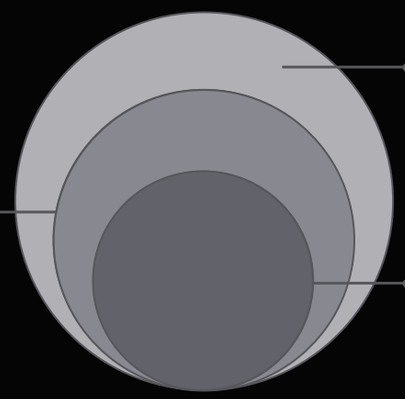
- Reduces tree depth and lookup time
- Frees extent tree nodes (less metadata)
- Improves sequential read performance
- Reduces journal transaction size

> Merging is attempted on every insert before a split is considered. This is the primary mechanism for keeping the tree compact.

mballoc: Buddy Cache Internals

The buddy cache is the heart of mballoc's free block tracking. It uses a binary buddy system to efficiently find contiguous free blocks of any power-of-2 size.

Buddy System Concept



Each order tracks free runs of 2^order blocks. Finding a run of N blocks requires scanning only the appropriate order level.

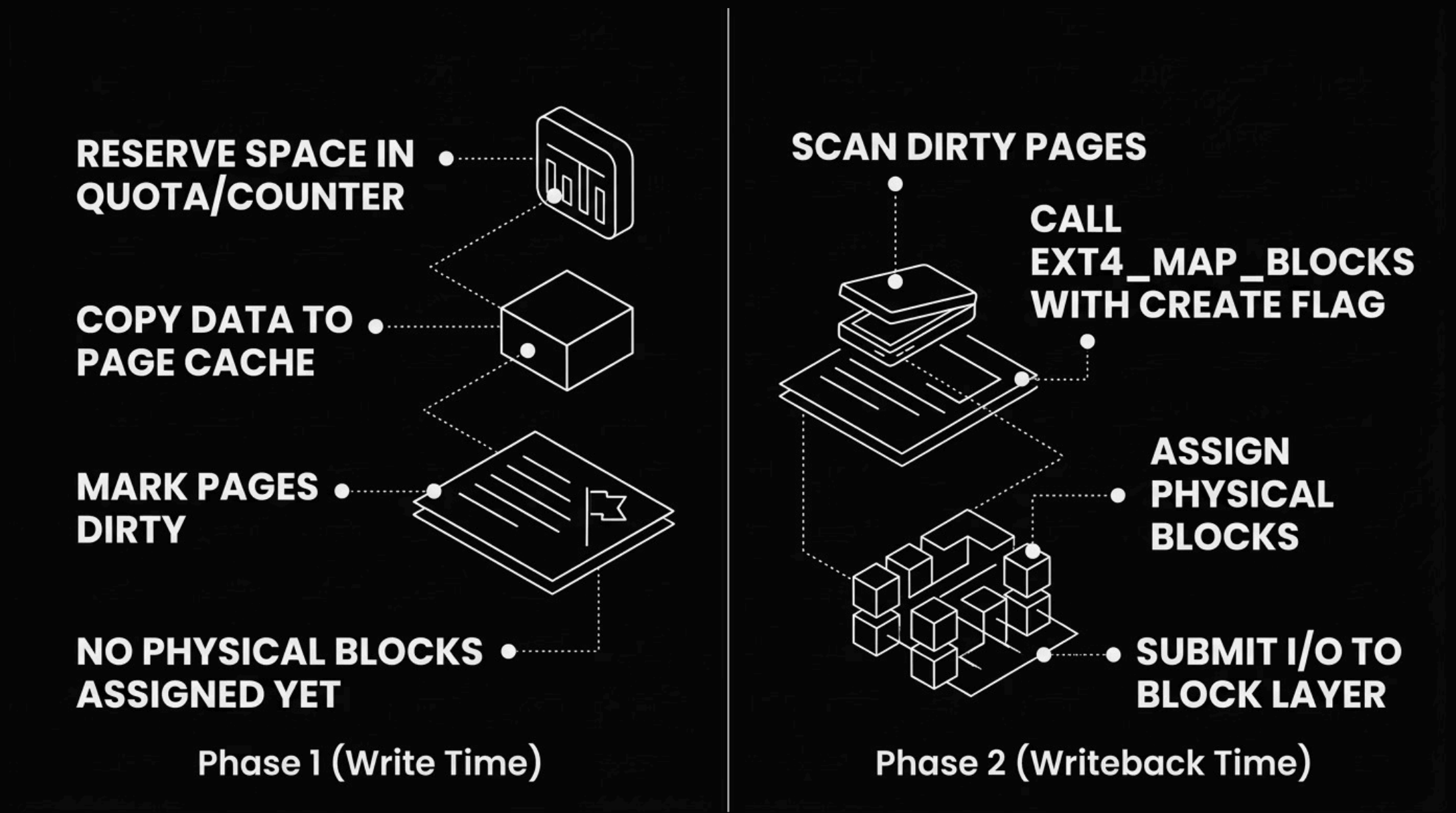
Group Info Fields

```
struct ext4_group_info {
    unsigned long  bb_free;
    /* Free blocks in group */

    unsigned long  bb_fragments;
    /* Number of free fragments */

    unsigned int   bb_largest_free_order;
    /* Largest contiguous free run (log2) */

    struct list_head bb_prealloc_list;
    /* Preallocated extents for this group */

    void         *bb_bitmap;
    /* Block bitmap cache */

    struct rw_semaphore alloc_sem;
    /* Protects allocation */
};
```

bb_largest_free_order allows mballoc to skip groups that cannot satisfy a request without scanning the full bitmap.

delalloc: Reservation vs Allocation

A common source of confusion is the distinction between *reserving* space and *allocating* blocks. Delayed allocation separates these two events in time.



Reservation Counters

`s_dirtyclusters_counter` tracks reserved-but-not-yet-allocated clusters. This is subtracted from free space to prevent over-commitment.

ENOSPC Risk

If the filesystem fills up between reservation and writeback, the write fails at writeback time — not at `write()` time. Applications must handle this.

Disabling delalloc

Mount with `nodatacow` or `nodelalloc` to disable. Useful for databases that manage their own I/O scheduling.

VFS Layer: The Glue Between Syscalls and Ext4

The VFS (Virtual Filesystem Switch) provides a uniform interface that allows the kernel to support multiple filesystem types simultaneously. Ext4 registers its operations with the VFS at mount time.

Registration at Mount

```
static int ext4_fill_super(struct super_block *sb,
                           void *data, int silent) {
    // Register superblock operations
    sb->s_op = &ext4_sops;

    // Register export operations
    sb->s_export_op = &ext4_export_ops;

    // Register quota operations
    sb->s_qcop = &ext4_qctl_operations;

    // Register xattr operations
    sb->s_xattr = ext4_xattr_handlers;
}
```

Key VFS Interfaces

super_operations

Filesystem-wide: alloc_inode, write_inode, put_super, statfs

inode_operations

Inode-level: create, lookup, link, unlink, mkdir, rename

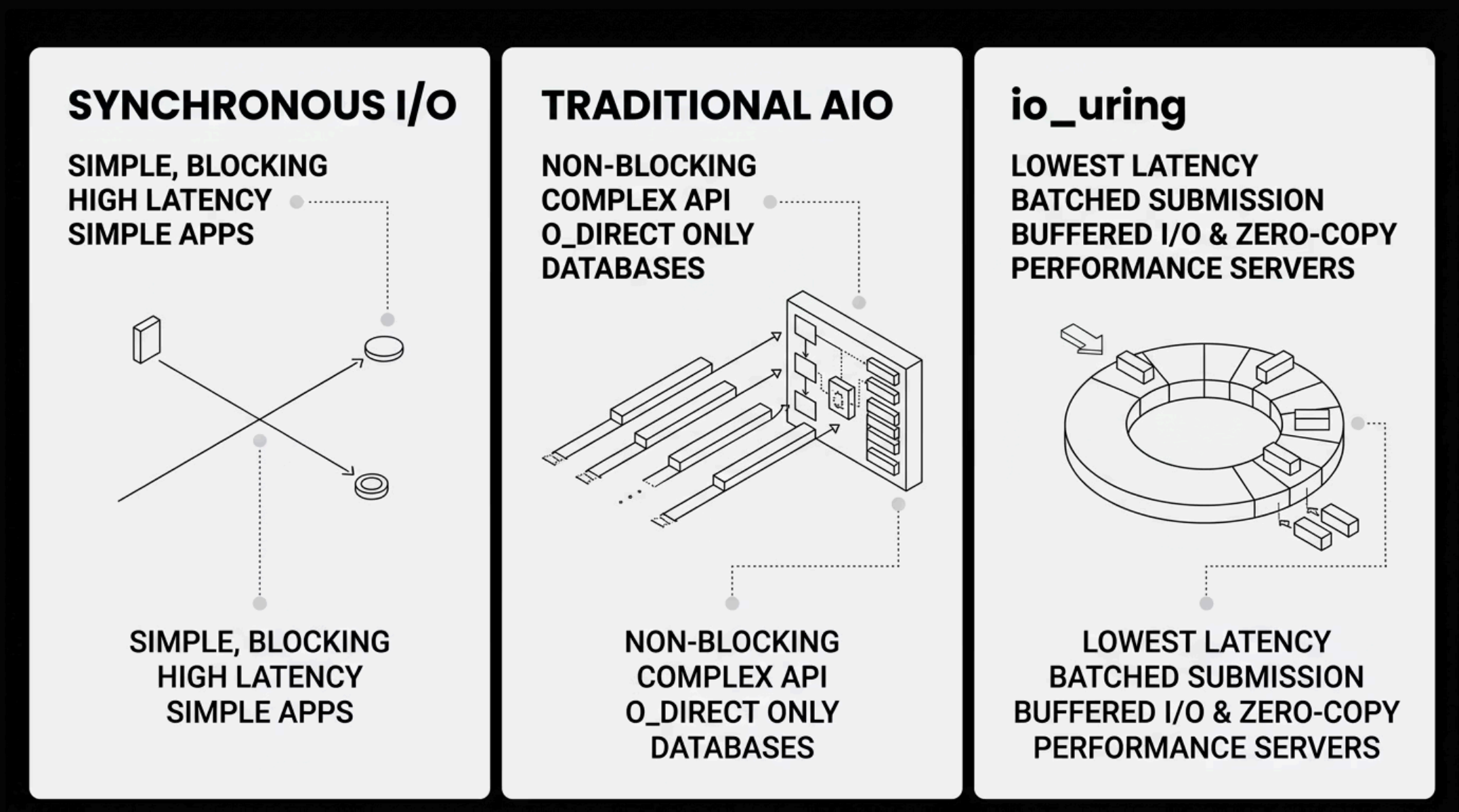
file_operations

File-level: read_iter, write_iter, mmap, fsync, fallocate

address_space_operations

Page cache: read_folio, writepages, write_begin, write_end

io_uring vs Traditional AIO: When to Choose



io_uring Advantages

- Lower latency than AIO — fewer syscalls via submission ring
- Batched submission — multiple operations per syscall
- Fixed buffers for zero-copy I/O
- Supports both buffered and direct I/O (unlike traditional AIO)

When to Choose AIO

- Legacy codebases already using `libaio`
- Environments where io_uring is not available (older kernels)
- Strict O_DIRECT-only workloads with simple patterns

> io_uring requires Linux 5.1+. For production systems on modern kernels, io_uring is generally preferred over traditional AIO.

Folio Interface: Why It Matters

The folio interface was introduced to allow the kernel to work with memory in units larger than a single 4KB page. This is increasingly important as hardware page sizes grow (e.g., 16KB on ARM64).

Problem with struct page

- Compound pages (multiple physically contiguous pages) were represented as an array of `struct page`, with the first page being the "head"
- Code had to constantly check whether a page was a head or tail, leading to bugs
- No type safety — a tail page could be accidentally passed where a head was expected

How Folio Solves This

- A `struct folio` always refers to the head of a compound page
- Type system prevents passing a tail page where a folio is expected
- Enables large page support (huge pages, 16KB pages on ARM64)
- Ext4 uses `folio_size()` to handle both 4KB and larger folios uniformly

The folio conversion in ext4 is ongoing. New code should use folio APIs; legacy buffer_head code is being gradually replaced.

Zoned Devices: Practical Implications for Ext4

Zone Types Recap

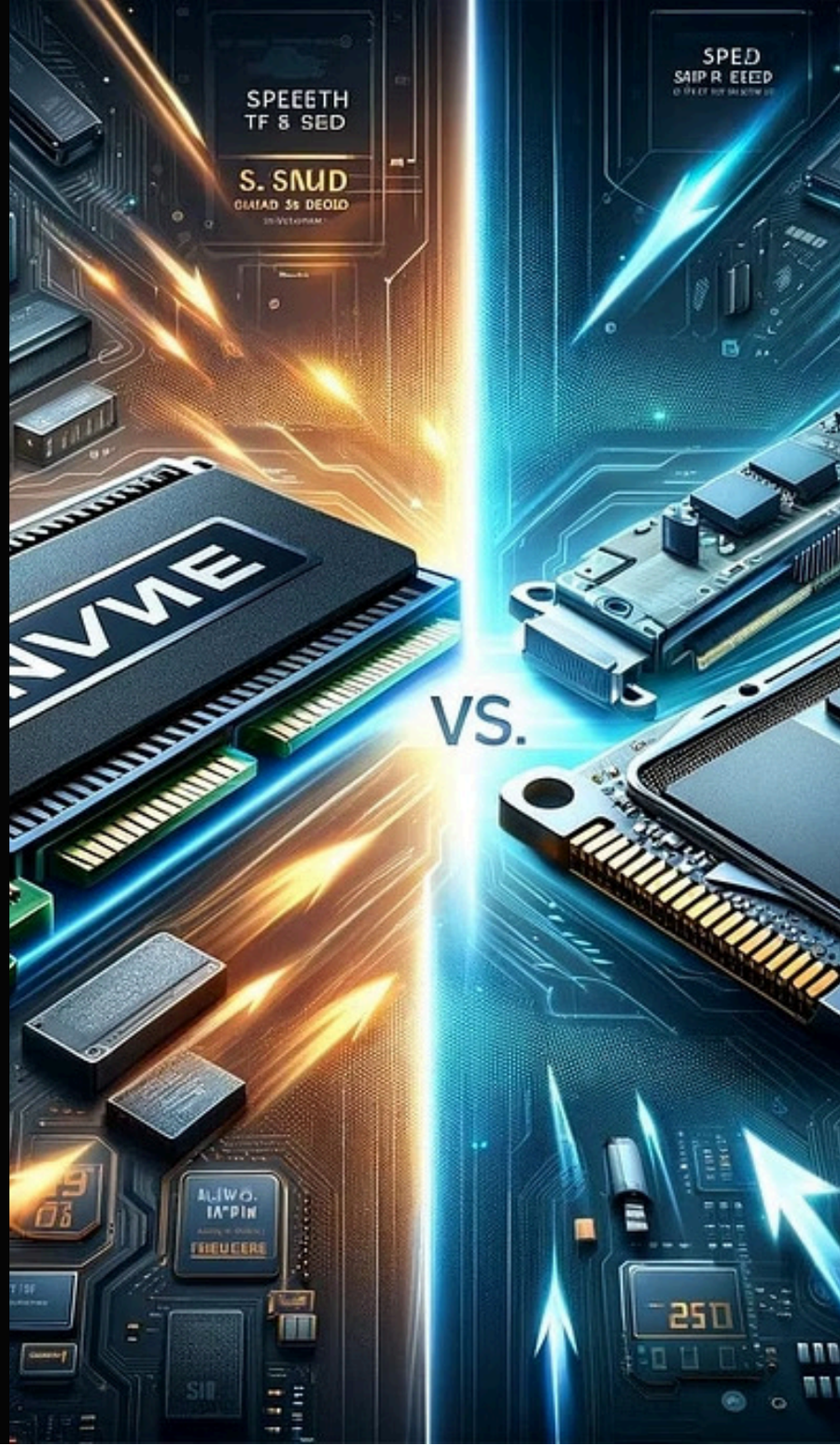
- **Conventional zones:** Random write capable — behave like a regular disk
- **Sequential write required (SWR):** Must write sequentially from the write pointer
- **Sequential write preferred (SWP):** Sequential writes preferred but not required

Ext4 Constraints on Zoned Devices

- Metadata must reside in conventional zones
- Journal must be in a conventional zone
- Data blocks in sequential zones must be written in order
- Zone reset required before reuse of sequential zones

Practical Commands

```
# Check zone layout
blkzone report /dev/sdb

# Reset a zone
blkzone reset -o 0x100000 -l 0x80000 /dev/sdb

# Format ext4 on zoned device
mkfs.ext4 -E lazy_itable_init=0 /dev/sdb

# Check zone write pointer
cat /sys/block/sdb/queue/zoned
```



blktrace Deep Dive: Reading the Output

Understanding blktrace output is essential for diagnosing I/O performance issues. Each event has a two-letter action code that describes what happened to the I/O request.

Key Action Codes

Code	Stage	Meaning
Q	Queue	I/O request queued to block layer
G	Get	Request structure allocated
M	Merge	Request merged with existing
I	Insert	Request inserted into device queue
D	Dispatch	Request dispatched to driver
C	Complete	I/O completed by device
P	Plug	Queue plugged (batching)
U	Unplug	Queue unplugged (flushed)

Interpreting btt Output

```
# btt produces latency breakdown:
# Q2G: Queue to Get (allocation time)
# G2I: Get to Insert (merge/plug time)
# I2D: Insert to Dispatch (scheduler time)
# D2C: Dispatch to Complete (device time)
# Q2C: Total latency (Q to C)

# Example btt output:
# ==================== All Devices ====================
# Q2G  0.000001  0.000002  0.000001
# G2I  0.000003  0.000010  0.000004
# I2D  0.000001  0.000005  0.000002
# D2C  0.000500  0.002000  0.000800
# Q2C  0.000505  0.002017  0.000807
```

D2C is the device service time. If D2C is high, the bottleneck is the storage device itself, not the kernel.

fio: Workload Generation Reference

fio is the standard tool for generating controlled I/O workloads. Understanding its key parameters is essential for the hands-on labs.

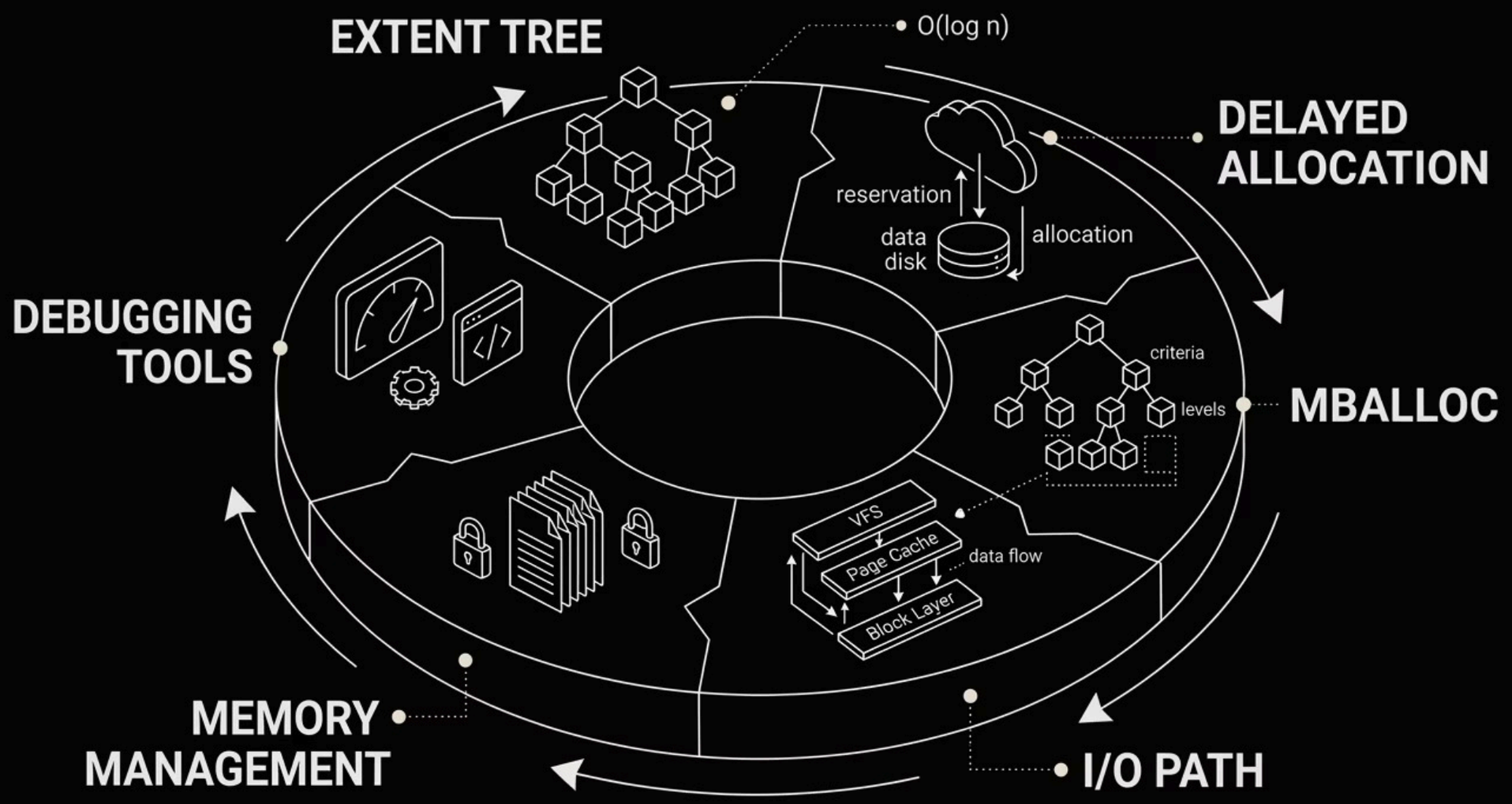
Essential Parameters

Parameter	Description
`--rw`	I/O pattern: read, write, randread, randwrite, randrw
`--bs`	Block size (e.g., 4k, 1M)
`--direct`	0 = buffered I/O, 1 = direct I/O
`--iodepth`	Queue depth for async I/O
`--ioengine`	libaio, io_uring, sync, psync
`--numjobs`	Number of parallel workers
`--runtime`	Duration in seconds
`--fsync`	Issue fsync every N writes

io_uring Workload Example

```
# io_uring async random read
fio --name=uring-randread \
 --filename=/mnt/test/testfile \
 --rw=randread \
 --bs=4k \
 --direct=1 \
 --ioengine=io_uring \
 --iodepth=64 \
 --numjobs=4 \
 --runtime=60 \
 --group_reporting \
 --output-format=json \
 --output=uring-randread.json

# Compare with libaio
fio --name=aio-randread \
 --filename=/mnt/test/testfile \
 --rw=randread \
 --bs=4k \
 --direct=1 \
 --ioengine=libaio \
 --iodepth=64 \
 --numjobs=4 \
 --runtime=60 \
 --group_reporting
```

Day 3 Complete: Key Concepts Recap



Day 3 covered the full data path from logical block allocation through the extent tree, delayed allocation, and multiblock allocator, all the way through the VFS and page cache to the block layer. These concepts form the foundation for the advanced tuning topics in Day 4.

End of Day 3

Questions?

Instructor

Chandrashekar Babu
training@chandrashekar.info

https://www.chandrashekar.info/
https://www.slashprog.com/

Tomorrow: Day 4

- Module 7: Advanced Ext4 Features
- Module 8: Performance Analysis & Tuning
- Hands-On: Encryption, mount flags, TRIM

Please review Day 3 concepts and prepare test devices (SSD, NVMe if available) and install encryption tools before Day 4.